

รายงาน

เรื่อง

**การพัฒนาโปรแกรม Network Application
และการออกแบบ Application-Layer Protocol**

MOEP

Mini Order Exchange Protocol

เสนอ

ผศ.ดร.สุขุมล กิตติสิน

ภาควิชาวิทยาการคอมพิวเตอร์ คณะวิทยาศาสตร์
มหาวิทยาลัยเกษตรศาสตร์

จัดทำโดย

ปิยากร ผลพานิช รหัสนักศึกษา 6710451054

รายงานนี้เป็นส่วนหนึ่งของรหัสวิชา 01418351

Project 1: Socket Programming

ภาคการศึกษาที่ 1 ปีการศึกษา 2569

1. วัตถุประสงค์ของโปรแกรม

1.1 โปรแกรมนี้คืออะไร

MOEP (Mini Order Exchange Protocol) เป็นโปรแกรมจำลองระบบตลาดหลักทรัพย์แบบ client-server ที่ประกอบด้วย exchange server จำนวน 1 ตัว ทำหน้าที่เป็นศูนย์กลาง และ trader client จำนวนหลายตัว ที่เชื่อมต่อเข้ามาซื้อขายหลักทรัพย์สัญลักษณ์เดียวกัน (ตัวอย่างในการพัฒนาใช้สัญลักษณ์ AAPL) โดยเปรียบเสมือนระบบซื้อขายหุ้นภายในองค์กรขนาดเล็กที่ trader หลายคนส่งคำสั่งซื้อขายเข้ามาพร้อมกัน และต้องเห็นความเคลื่อนไหวของราคาแบบเรียลไทม์ตรงกันทุกคน

1.2 โปรแกรมนี้ใช้สำหรับทำอะไร

- รับคำสั่งซื้อขาย (order) จาก trader แต่ละคน แล้วนำไปจับคู่ (matching) กับคำสั่งฝั่งตรงข้ามที่มีอยู่ในระบบตามหลัก *price-time priority*
- รักษา order book กลางที่เป็นความจริงหนึ่งเดียว (single source of truth) ของราคาซื้อ-ขายที่ดีที่สุด ณ ขณะนั้น
- กระจายข้อมูลตลาด (market data) และผลการจับคู่ (trade) ให้ trader ทุกคนเห็นพร้อมกันแบบเรียลไทม์
- รองรับการยกเลิกคำสั่งที่ยังไม่ถูกจับคู่ และแจ้งสถานะกลับให้ trader ทราบผ่าน status code ที่ชัดเจน

2. คุณลักษณะของ Application (Application Characteristics)

คุณลักษณะ	รายละเอียด
Many-to-one-to-many	Trader หลายคนส่งคำสั่งเข้า server ตัวเดียว (many-to-one) จากนั้น server กระจายสถานะตลาดที่อัปเดตแล้วกลับไปให้ trader ทุกคนพร้อมกัน (one-to-many) ในรอบเดียว
Central authoritative state	Order book กลางบนฝั่ง server เป็นความจริงหนึ่งเดียวของระบบ ฝั่ง client ไม่เก็บ state การจับคู่เอง แต่รับรู้สถานะทั้งหมดผ่านข้อความที่ server ส่งมาเท่านั้น
Continuous, session-persistent	การเชื่อมต่อไม่มีลักษณะเป็นรอบ (round) แต่ค้าง connection ไว้ตลอด session เดียว โดย client ส่ง REGISTER เพียงครั้งเดียวตอนเข้าสู่ระบบ แล้วสามารถส่งคำสั่งซื้อขายต่อได้ ไม่จำกัดโดยไม่ต้อง register ซ้ำ
Dual-channel ต่อ client หนึ่งคน	Trader แต่ละคนถือช่องทางสื่อสาร 2 ช่องพร้อมกัน คือ TCP connection สำหรับส่งคำสั่งซื้อขาย และ UDP socket ของตนเอง (ephemeral port ที่ระบบปฏิบัติการสุ่มให้) สำหรับรับข้อมูลตลาด
Self-healing market data feed	หาก packet ข้อมูลตลาดบางส่วนสูญหายระหว่างทาง client จะตรวจพบได้เองจากเลขลำดับ (sequence number) ที่กระโดดข้าม และสามารถกู้คืนข้อมูลที่หายไปได้เองผ่านกลไก gap-fill โดยผู้ใช้ไม่ต้องดำเนินการใดๆ
ใช้ client library เดียวกันสำหรับ 2 ส่วนติดต่อผู้ใช้	ทั้ง terminal CLI และ web dashboard ใช้ไลบรารีสื่อสารกับ protocol ชุดเดียวกันทุกตัวอักษร ไม่มีการดัดแปลง wire format เพื่อรองรับหน้าเว็บเป็นการเฉพาะ ทำให้พฤติกรรมของทั้งสองส่วนติดต่อผู้ใช้ สอดคล้องกันเสมอ

3. การเลือก Service Model ของ Transport Layer

สรุปคำตอบ: MOEP เลือกใช้ transport ทั้งสองแบบร่วมกัน คือ TCP สำหรับคำสั่งซื้อขาย (order entry) และ UDP สำหรับข้อมูลตลาด (market data) โดยแบ่งตามประเภทของ traffic ไม่ใช่เลือกเพียงแบบใดแบบหนึ่งสำหรับทั้งโปรแกรม

3.1 เหตุผลที่ traffic สองประเภทต้องใช้ transport ต่างกัน

MOEP มี traffic สองลักษณะที่มีข้อกำหนด (requirement) แตกต่างกันโดยพื้นฐาน จึงไม่ควรถูกบังคับ ให้ใช้ transport model เดียวกัน ดังแสดงในตารางเปรียบเทียบต่อไปนี้

หัวข้อเปรียบเทียบ	Order Entry	Market Data
ทิศทางการสื่อสาร	1 client → server	server → clients ทั้งหมด
ความถี่ในการส่ง	ต่ำ (ตามจังหวะที่ trader สั่งซื้อขาย)	สูง (ทุกครั้งที่ order book เปลี่ยนแปลง)
ยอมให้ ข้อมูล สูญหาย ได้หรือไม่	ยอมไม่ได้ — ความถูกต้องเทียบเท่าเงินจริง	ยอมได้ชั่วคราว ขอเพียงกู้คืนได้ภายหลัง
ความสำคัญของลำดับ	สำคัญมาก (กำหนด price-time priority)	สำคัญน้อยกว่า เพราะ client มักสนใจค่าล่าสุด

3.2 เหตุผลที่เลือก TCP สำหรับ Order Entry

Correctness เทียบเท่าเงินจริง

คำสั่งซื้อขายที่สูญหายหรือถูกส่งซ้ำถือเป็นข้อผิดพลาดทางการเงิน ไม่ใช่เพียงข้อบกพร่องด้าน UX ดังนั้นช่องทางนี้จึงต้องการการรับประกันการส่งถึงปลายทางอย่างครบถ้วน (reliable delivery) ซึ่งเป็นคุณสมบัติพื้นฐานของ TCP

การส่งถึงตามลำดับ (in-order delivery)

ลำดับที่คำสั่งซื้อขายมาถึง server คือปัจจัยที่ใช้ตัดสิน price-time priority โดยตรง หากลำดับของคำสั่งถูกสลับระหว่างทาง ผลการจับคู่จะผิดพลาดทันที TCP รับประกันว่าข้อมูลจะถูกส่งมอบให้ชั้น application ตามลำดับเดียวกับที่ถูกส่งออกไป

การใช้ประโยชน์จาก connection state

การที่ TCP เป็น connection-oriented ทำให้ connection ที่ค้างไว้ตลอด session สามารถใช้แทนตัวตนของ trader คนนั้นได้เลย โดยไม่ต้องแนบ token ยืนยันตัวตนซ้ำในทุกคำสั่ง และ server สามารถรู้ได้ทันทีเมื่อ client หลุดการเชื่อมต่อ

3.3 เหตุผลที่เลือก UDP สำหรับ Market Data

หลีกเลี่ยง head-of-line blocking

หากใช้ TCP กับข้อมูลตลาด เมื่อ update ลำดับที่ 41 สูญหายระหว่างทาง update ลำดับที่ 42 และ 43 ที่มาถึงภายหลังจะต้องรอ (block) อยู่ในบัฟเฟอร์จนกว่า update 41 จะถูกส่งซ้ำสำเร็จ ทั้งที่ในความเป็นจริง client ต้องการเพียงราคาล่าสุดเท่านั้น การรอเช่นนี้จึงเป็นความล่าช้าที่ไม่จำเป็น

การเรียก sendto() ไม่มีการ block

Server สามารถกระจายข้อมูลตลาดให้ client จำนวนมากได้โดยไม่ต้องรอการตอบรับ (ACK) จากฝ่ายใดเลย จึงไม่ทำให้ thread ที่กำลังประมวลผลการจับคู่คำสั่งซื้อขายต้องหยุดรอ

รองรับการขยายเป็น multicast ได้ในอนาคต

คุณสมบัติ connectionless ของ UDP เปิดทางให้สามารถขยายจาก unicast fan-out ในปัจจุบัน ไปสู่ IP multicast ได้ในอนาคตหากจำนวน client เพิ่มขึ้นมาก ซึ่ง TCP ไม่สามารถทำได้โดยธรรมชาติ

ในการพัฒนาปัจจุบัน การกระจายข้อมูลยังดำเนินการด้วยการวนเรียก sendto() ทีละ client (unicast fan-out) ยังไม่ใช่ IP broadcast หรือ multicast จริง

3.4 สรุปหลักการเลือก Transport

หลักการออกแบบของ MOEP ไม่ใช่การเลือกระหว่าง “ความเร็ว” กับ “ความแม่นยำ” เพียงอย่างเดียว แต่เป็นการวางความแม่นยำไว้ในจุดที่จำเป็นต้องแม่นยำ (คำสั่งซื้อขาย) และวางความเร็วไว้ในจุดที่ความเร็วให้ผลลัพธ์ที่คุ้มค่ากว่า (ข้อมูลตลาด) การแบ่ง transport ตามประเภทของ traffic เช่นนี้ทำให้ระบบได้ประโยชน์ จากทั้งสองฝั่งพร้อมกัน

4. การออกแบบ Application-Layer Protocol: MOEP

4.1 ชื่อโปรโตคอลและภาพรวม

โปรโตคอลที่ออกแบบขึ้นสำหรับโปรแกรมนี้มีชื่อว่า **MOEP (Mini Order Exchange Protocol)** เป็นโปรโตคอลระดับ application layer ที่ทำงานอยู่บนสอง transport พร้อมกัน โดยแบ่งข้อความออกเป็น 3 กลุ่มตามทิศทางการสื่อสาร ได้แก่ (1) Client → Server ผ่าน TCP (2) Server → Client เฉพาะ connection นั้นผ่าน TCP และ (3) Server → Clients ทั้งหมด ผ่าน UDP fan-out ทุกข้อความเข้ารหัสในรูปแบบ JSON

4.2 Framing และ Port

Framing

- **TCP:** 1 ข้อความต่อ 1 บรรทัด โดยคั่นแต่ละ JSON message ด้วยอักขระ newline (\n) เนื่องจาก TCP เป็น byte stream ที่ไม่มีขอบเขตข้อความในตัวเอง จึงต้องกำหนดตัวคั่นขึ้นเอง
- **UDP:** 1 datagram ต่อ 1 JSON message เนื่องจาก UDP รักษาขอบเขตของข้อความให้อยู่แล้วโดยธรรมชาติ จึงไม่จำเป็นต้องมีตัวคั่นเพิ่มเติม

Port

- **TCP:** ใช้ port คงที่หมายเลข 5000 โดย trader ทุกคนเชื่อมต่อเข้ามาที่ port เดียวกันนี้
- **UDP:** ไม่มี port กลาง client แต่ละคน bind port ของตนเอง (เป็น ephemeral port ที่ระบบปฏิบัติการสุ่มให้) แล้วแจ้งหมายเลข port นั้นให้ server ทราบผ่าน field udp_port ในข้อความ REGISTER

4.3 ข้อความจาก Client ไปยัง Server (ผ่าน TCP)

REGISTER — เข้าสู่ตลาด (ส่งครั้งเดียวต่อ session)

```
{"type": "REGISTER", "udp_port": <int>}
```

ORDER_NEW — ส่งคำสั่งซื้อขาย (ส่งซ้ำได้ไม่จำกัด)

```
{"type": "ORDER_NEW", "symbol": "AAPL", "client_order_id": <str>,
  "side": "BUY" | "SELL", "price": <float>, "qty": <int>}
```

ORDER_CANCEL — ยกเลิกคำสั่งที่ยังไม่ถูกจับคู่หมด

```
{"type":"ORDER_CANCEL","client_order_id":<str>}
```

GAP_FILL_REQUEST — ขอข้อมูลตลาดที่หายไปคืน

```
{"type":"GAP_FILL_REQUEST","from_seq":<int>,"to_seq":<int>}
```

4.4 ข้อความจาก Server กลับไปยัง Client (ผ่าน TCP เฉพาะ connection นั้น)**REGISTER_ACK**

```
{"type":"REGISTER_ACK","status_code":200,"status_phrase":"OK",
  "client_id":<str>,"symbol":"AAPL"}
```

ORDER_ACK (จับคู่เต็มจำนวนทันที)

```
{"type":"ORDER_ACK","status_code":201,"status_phrase":"MATCHED","order_id":<int>,
  "filled_qty":<int>,"remaining_qty":0,"trade_count":<int>}
```

ORDER_ACK (จับคู่ได้บางส่วน ส่วนที่เหลือขึ้น book)

```
{"type":"ORDER_ACK","status_code":202,"status_phrase":"PARTIAL_FILL", ...}
```

CANCEL_ACK

```
{"type":"CANCEL_ACK","status_code":204,"status_phrase":"CANCELLED","order_id":<int>}
```

GAP_FILL_RESPONSE

```
{"type":"GAP_FILL_RESPONSE","status_code":200,"messages":[...]}
```

4.5 ข้อความจาก Server กระจายให้ Client ทั้งหมด (ผ่าน UDP fan-out)

MARKET_DATA — ส่งทุกครั้งที่ order book เปลี่ยนแปลง

```
{ "type": "MARKET_DATA", "seq": <int>, "best_bid": <float>, "best_ask": <float>,
  "bids": [ { "price": ..., "qty": ... }, "asks": [ { "price": ..., "qty": ... } ] }
```

MARKET_DATA แต่ละข้อความเป็น snapshot เต็มของ order book 5 ระดับราคาบนสุด (top-5) ไม่ใช่ delta ทำให้ client ที่เพิ่งเชื่อมต่อเข้ามาใหม่ หรือเพิ่งพลาด packet ก่อนหน้า สามารถเห็นภาพตลาด ล่าสุดครบถ้วนได้จากข้อความเพียงข้อความเดียว โดยไม่ต้องอาศัยประวัติก่อนหน้า

TRADE — ส่งเมื่อเกิดการจับคู่คำสั่งซื้อขายสำเร็จ

```
{ "type": "TRADE", "seq": <int>, "price": <float>, "qty": <int>, "side": "BUY" | "SELL" }
```

MARKET_DATA และ TRADE ใช้ตัวนับ seq ชุดเดียวกันต่อเนื่องกัน เพื่อให้ client ตรวจสอบข้อมูลที่ขาดหายได้จาก stream เพียงชุดเดียว ไม่ต้องแยกตรวจสอบสองระบบ

4.6 กลไกความน่าเชื่อถือของข้อมูลตลาด (Reliability over UDP)

เนื่องจาก UDP ไม่รับประกันการส่งถึงปลายทาง MOEP จึงออกแบบกลไกเสริมความน่าเชื่อถือไว้ที่ชั้น application ดังนี้

- **Sequence numbering:** ข้อความ UDP ทุกประเภท (ทั้ง MARKET_DATA และ TRADE) ใช้ตัวนับ seq ตัวเดียวกันเรียงต่อเนื่อง client จึงตรวจพบได้ทันทีเมื่อค่าที่ได้รับไม่ต่อเนื่อง กับค่าที่คาดหวัง (เช่น expected=41 แต่ได้รับ got=43)
- **Gap fill ผ่านช่องทางที่เชื่อถือได้:** เมื่อ client ตรวจพบว่าข้อมูลขาดหาย จะส่ง GAP_FILL_REQUEST(from_seq to_seq) กลับไปทาง TCP แทนที่จะรอทาง UDP จากนั้น server จะค้นหาข้อความที่ขาดหายจาก history buffer (เก็บย้อนหลัง 200 ข้อความล่าสุด) แล้วส่งกลับให้ครบทุกข้อความ
- **กรณีข้อมูลหลุดจาก buffer แล้ว:** หาก seq ที่ client ร้องขอหลุดออกจาก history buffer ไปแล้ว server จะตอบกลับด้วยรหัสสถานะ 410 GONE พร้อมระบุ missing_seq ให้ทราบ

ความน่าเชื่อถือของ MOEP ไม่ได้มาจาก transport เพียงชนิดเดียว แต่เกิดจากการออกแบบให้ TCP และ UDP ทำงานประกอบกัน — UDP รับผิดชอบความเร็วของข้อมูลปกติ ส่วน TCP รับผิดชอบการกู้คืนเมื่อเกิดข้อผิดพลาด

4.7 สถานะของคำสั่งซื้อขาย (Order State)

คำสั่งซื้อขายแต่ละรายการมีสถานะเปลี่ยนแปลงได้ตามเงื่อนไขดังตาราง

สถานะปัจจุบัน	เหตุการณ์	สถานะถัดไป
(ORDER_NEW เข้าสู่ book)	คำสั่งเข้าสู่ order book	RESTING
RESTING	จับคู่ได้บางส่วน	PARTIALLY_FILLED
PARTIALLY_FILLED	จับคู่จนหมด	FILLED
RESTING	ยกเลิก	CANCELLED
PARTIALLY_FILLED	ยกเลิก	CANCELLED

คำสั่งที่มีสถานะ FILLED หรือ CANCELLED แล้วถือเป็นสถานะสิ้นสุด (terminal state) ไม่สามารถยกเลิกซ้ำได้อีก หากมีการร้องขอ server จะตอบกลับด้วยรหัสสถานะ 404 NOT_FOUND

4.8 รหัสสถานะ (Status Code) ทั้งหมดในโปรโตคอล

Code	Phrase	ความหมาย
200	OK	สำเร็จทั่วไป (register สำเร็จ หรือ order ถูกวางบน book)
201	MATCHED	คำสั่งจับคู่ได้เต็มจำนวนทันที
202	PARTIAL_FILL	จับคู่ได้บางส่วน ส่วนที่เหลือค้างอยู่บน book
204	CANCELLED	ยกเลิกคำสั่งสำเร็จ
400	BAD_REQUEST	รูปแบบ JSON ผิด / ค่า side ไม่ถูกต้อง / ไม่รู้จัก message type
403	REJECTED	ราคาหรือจำนวนไม่ถูกต้อง (มีค่าน้อยกว่าหรือเท่ากับ ศูนย์)
404	NOT_FOUND	ยกเลิกคำสั่งที่ไม่มีอยู่จริงหรือสิ้นสุดสถานะไปแล้ว
410	GONE	ขอ gap-fill สำหรับ seq ที่หลุดจาก history buffer ไปแล้ว
500	INTERNAL_ERROR	กำหนดไว้ใน spec สำหรับกรณีข้อผิดพลาดของระบบ

4.9 ตัวอย่าง Message Flow

(1) การเข้าตลาดและการจับคู่คำสั่งซื้อขายปกติ

```
Client1 -TCP-> {"type":"REGISTER","udp_port":5001}
Server   -TCP-> {"status_code":200,"status_phrase":"OK","client_id":"c_101"}

Client2 -TCP-> {"type":"REGISTER","udp_port":5002}
Server   -TCP-> {"status_code":200,"status_phrase":"OK","client_id":"c_102"}

Client1 -TCP-> {"type":"ORDER_NEW","side":"BUY","price":214.41,"qty":100}
Server   -TCP-> {"status_code":200,"status_phrase":"OK","order_id":1}
Server   -UDP-> MARKET_DATA seq=1 bids=[{214.41,100}] asks=[]

Client2 -TCP-> {"type":"ORDER_NEW","side":"SELL","price":214.41,"qty":100}
Server   -TCP-> {"status_code":201,"status_phrase":"MATCHED","order_id":2}
Server   -UDP-> TRADE seq=2 price=214.41 qty=100
Server   -UDP-> MARKET_DATA seq=3 bids=[] asks=[]
```

(2) กรณี packet สูญหายและกู้คืนผ่าน gap fill

```
Client1 <-- UDP: seq=1, seq=3          (seq=2 สูญหายระหว่างทาง)
Client1 expected=2, got=3              --> ตรวจพบว่าข้อมูลขาดหาย

Client1 -TCP-> {"type":"GAP_FILL_REQUEST","from_seq":2,"to_seq":2}
Server   -TCP-> {"type":"GAP_FILL_RESPONSE","status_code":200,
                "messages":[{"type":"TRADE","seq":2, ...}]}
```

Client1 [กู้คืนสำเร็จ] (stream ต่อเนื่องครบ seq 1, 2, 3)

สังเกตว่ากระบวนการกู้คืนทั้งหมดวิ่งอยู่บน TCP เท่านั้น ฝ่าย UDP ไม่จำเป็นต้องรับรู้หรือจัดการ เรื่องนี้เลย ซึ่งตอกย้ำหลักการที่ว่าความน่าเชื่อถือของระบบเกิดจากการออกแบบให้ transport สองชนิด ทำงานประกอบกัน

5. สรุป

- MOEP แบ่ง transport **ตามประเภทของ traffic** ไม่ใช่ตามตัวแอปพลิเคชัน โดยใช้ TCP สำหรับคำสั่งซื้อขาย และ UDP สำหรับข้อมูลตลาด
- เลือก TCP สำหรับ order entry เพราะคำสั่งซื้อขายต้องถูกต้องและเรียงลำดับ — ความถูกต้องในที่นี้เทียบเท่ากับมูลค่าเงินจริง
- เลือก UDP สำหรับ market data เพราะข้อมูลราคาต้องสดใหม่เสมอ และไม่ควรรยอมให้ packet เก่า ที่สูญหายไปบล็อก packet ใหม่ที่มาถึงภายหลัง
- จุดที่ UDP อาจทำข้อมูลสูญหายได้รับการแก้ไขด้วยกลไก **sequence number ร่วมกับ gap-fill ผ่าน TCP** ทำให้ระบบได้รับประโยชน์จากทั้งความเร็วของ UDP และความน่าเชื่อถือของ TCP พร้อมกัน โดยไม่ต้องเลือกข้างใดข้างหนึ่งแล้วรับผลเสียทั้งหมด

โดยสรุป MOEP เป็นตัวอย่างของการออกแบบ application-layer protocol ที่พิจารณาคุณสมบัติของข้อมูล แต่ละประเภทเป็นหลัก แล้วจึงเลือก transport service model ให้เหมาะสมกับข้อมูลนั้น มากกว่าการยึดติดกับ transport เพียงชนิดเดียวสำหรับทั้งโปรแกรม